

PREDICTION MARKET

Task 1a - Contract Architecture & Relationship Design

SC4053 Blockchain Technology - Development Project

Document purpose

Define the proposed on-chain architecture for the Prediction Market MVP before implementation. This design fixes the responsibilities, relationships, interfaces, data ownership, events, and access rules for MarketFactory, Market, and OutcomeToken so Task 1b and the frontend can build against stable interfaces.

1. Source Requirements

The course brief requires a decentralized prediction market where users can create, trade, and resolve speculative markets. For market creation, each market must include outcome options, timestamps, and an arbitrator identity. The project task breakdown assigns Task 1a to the architecture and market-creation foundation.

Source requirement	How this design addresses it
Market Creation	MarketFactory.createMarket(...) deploys and registers a new Market.
Outcome options	Market stores an ordered list of outcome labels; token IDs use the same zero-based index.
Timestamps	Market stores startTime and endTime and validates their ordering.
Arbitrator identity	Market stores a designated arbitrator address for later Task 1b resolution.
Trading & Minting	Reserved for Task 1b; OutcomeToken is included now so the architecture is ready.
Resolution & Payouts	Reserved for Task 1b; the Market owns this lifecycle later.

Source basis: SC4053 Development Project brief, Option 4: Prediction Market (p. 7), and Prediction Market Project - Task Breakdown, Task 1a.

2. Proposed Contract Architecture

The proposed MVP uses three contracts. The Factory is the registry and creation entry point, each Market owns one market lifecycle, and each Market controls its own ERC-1155 OutcomeToken contract.

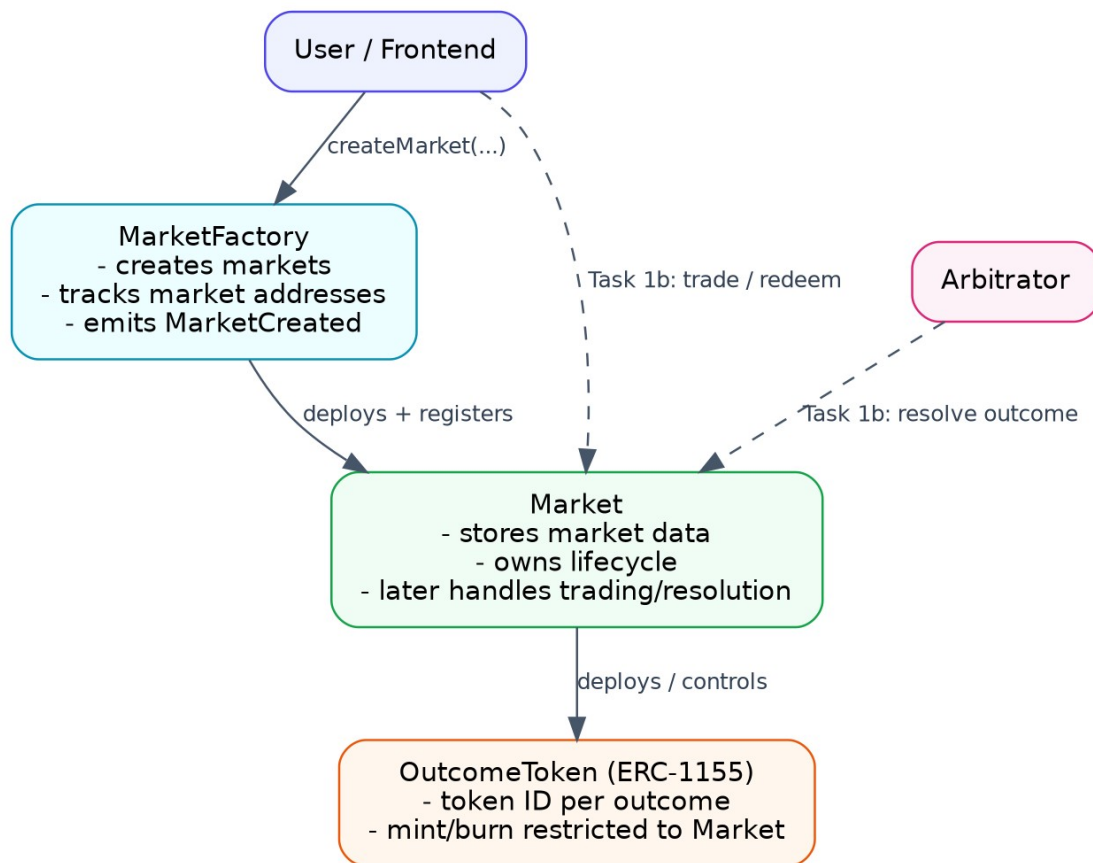


Figure 1. Proposed Task 1a contract relationships. Dashed interactions belong to Task 1b.

2.1 Relationship Summary

- User / Frontend -> MarketFactory: create a new market.
- MarketFactory -> Market: deploy a dedicated Market contract and store its address.
- Market -> OutcomeToken: deploy and control the token contract used for outcome shares.
- Arbitrator -> Market: later submits the final result in Task 1b.
- User -> Market: later trades and redeems through the Market in Task 1b.

Key architecture decision

Use one OutcomeToken contract per Market. This keeps token IDs simple (0, 1, 2, ... within that market), makes mint/burn authorization straightforward, and avoids a global token-ID namespace. This is a proposed design choice, not a requirement stated by the course brief.

3. Contract Responsibilities

3.1 MarketFactory.sol

Purpose: the single creation and discovery entry point for markets.

- Accept the market creation parameters from a user.
- Deploy a new Market contract.
- Pass the creator address (msg.sender) and arbitrator into the Market constructor.
- Store the new Market address in a registry.
- Emit MarketCreated so the frontend can discover new markets from logs.

Should not do

MarketFactory should not contain AMM pricing, user share balances, resolution logic, or payout logic. Those belong to each Market and Task 1b.

3.2 Market.sol

Purpose: represent one prediction question and own its complete lifecycle.

- Store question, outcome labels, creator, arbitrator, startTime, endTime, and resolution state.
- Expose read functions used by the frontend.
- Deploy or reference the market-specific OutcomeToken.
- Be the only contract allowed to mint/burn outcome shares.
- Later in Task 1b: own AMM trading, resolution, redemption, and expiry/timeout handling.

3.3 OutcomeToken.sol

Purpose: represent conditional outcome shares using ERC-1155.

- One token ID corresponds to one market outcome.
- Example for a binary market: token ID 0 = YES, token ID 1 = NO.
- Only the owning Market contract can mint or burn shares.
- Users can hold balances and use the standard ERC-1155 balanceOf interface.

3.4 State Ownership

Contract	Core state	Reason
MarketFactory	markets[]	Registry/discovery only.
Market	question, outcomes, creator, arbitrator, times, resolved, winningOutcome, outcomeToken	All market-specific state stays together.
OutcomeToken	market authority + ERC-1155 balances	Token ledger is separated from market logic.

4. Market Creation Flow

1. User calls `MarketFactory.createMarket(...)`
2. `MarketFactory` validates / forwards creation parameters.
3. `MarketFactory` deploys new `Market(..., creator = msg.sender, ...)`.
4. `Market` constructor validates its immutable creation data.
5. `Market` deploys its `OutcomeToken` contract.
6. `MarketFactory` stores `address(newMarket)` in `markets[]`.
7. `MarketFactory` emits `MarketCreated(...)`.
8. Frontend can read the `Market` and `OutcomeToken` addresses.

4.1 Creation Parameters

Parameter	Example	Validation
question	"Will ETH exceed \$5,000 by 31 Dec 2026?"	Non-empty.
outcomes	["YES", "NO"]	At least 2; recommended: no empty labels and no duplicates.
startTime	Unix timestamp	Must be before <code>endTime</code> .
endTime	Unix timestamp	Must be after <code>startTime</code> ; recommended: future time.
arbitrator	Ethereum address	Must not be <code>address(0)</code> .

Validation placement

The `Market` constructor should defend its own invariants even if the `Factory` also validates them. That way, the `Market` cannot be deployed into an invalid state if the creation path changes later.

4.2 Access-Control Matrix

Action	User	Market	Arbitrator
Create market	Allowed	N/A	N/A
Mint outcome shares	Not directly	Allowed	Not directly
Burn outcome shares	Not directly	Allowed	Not directly
Resolve final outcome (Task 1b)	No	Enforces rule	Allowed
Trade / redeem (Task 1b)	Allowed through Market	Executes	No

5. Interface Draft for Task 1a

These signatures are the proposed interface contract between the smart-contract and frontend tracks. They can be adjusted during implementation, but the team should agree on them early because Task 2a depends on the ABI.

5.1 MarketFactory

```
function createMarket(
    string calldata question,
    string[] calldata outcomes,
    uint256 startTime,
    uint256 endTime,
    address arbitrator
) external returns (address market);

function getMarkets() external view returns (address[] memory);
function getMarketCount() external view returns (uint256);
```

5.2 Market

```
question() -> string
creator() -> address
arbitrator() -> address
startTime() -> uint256
endTime() -> uint256
resolved() -> bool
winningOutcome() -> uint256
outcomeToken() -> address
getOutcomes() -> string[]
getOutcomeCount() -> uint256
```

5.3 OutcomeToken

```
mint(address to, uint256 outcomeId, uint256 amount) // only Market
burn(address from, uint256 outcomeId, uint256 amount) // only Market
balanceOf(address account, uint256 id) -> uint256 // ERC-1155
```

5.4 Task 1a Event

```
event MarketCreated(
    address indexed market,
    address indexed creator,
    address indexed arbitrator,
    string question,
    uint256 startTime,
    uint256 endTime
);
```

6. Scope Boundary and Next Step

Included in Task 1a

- MarketFactory, Market, and OutcomeToken skeletons.
- Market creation and creation-time validation.
- Market registry and MarketCreated event.
- Outcome labels, timestamps, creator, and arbitrator state.
- Stable read interfaces for the frontend.
- Basic unit tests for successful and invalid market creation.

Deferred to Task 1b

- AMM pricing model (LMSR or bonding curve).
- Buying/selling and dynamic odds.
- Oracle / arbitrator resolution rules.
- Dispute, staking, or timeout mechanics if implemented.
- Winner redemption and payout accounting.
- Trading-specific edge cases and rounding tests.

Definition of done for this design

The team can explain what each contract owns, who may call sensitive functions, how a market is created, which interfaces the frontend can rely on, and which features are intentionally postponed to Task 1b.

7. Design Rationale / AI Audit Note

This document is an AI-assisted proposed architecture and should be retained as part of the project history. The course requires an AI Audit Report covering the initial AI-generated architecture, problems discovered, validation/testing, and improvements. During implementation, record any design changes and why they were necessary.

Design choice to review	What to test / challenge
One OutcomeToken per Market	Deployment gas vs. simpler authorization and token IDs.
onlyMarket mint/burn	Verify users cannot mint or burn shares directly.
Creation validation	Test empty question/outcomes, duplicate labels, invalid times, and zero arbitrator.
Factory registry	Check market addresses are stored once and events match stored data.
Future resolution access control	Verify only the designated arbitrator can resolve and cannot resolve twice.

Prepared from the provided project brief and task breakdown. Proposed architecture decisions are clearly identified and should be validated through implementation, unit tests, static analysis, and testnet execution.